CO1 - AT1

Course Code: MLA0201 **Name:** Siva Balan K

Course Name: Fundamentals of Machine Learning **Reg.no:** 192572037

Q1.

Spam Email Prediction Using Probability Distribution

Problem Description

The system predicts whether an email is spam or legitimate. TF-IDF converts the email text into numerical features, and Multinomial Naive Bayes calculates the probability of each class.

- 0 – Not Spam
- 1 – Spam

Dataset Description

The Spam Email Classification Dataset by **Puru Singhvi** contains approximately **83,448 labelled emails** with the columns text and label.

Dataset file: spam-email-classification.csv
Source: Spam Email Classification Dataset – Kaggle

Algorithm

1. Load and clean the email dataset.
2. Remove missing and duplicate emails.
3. Split the data into training and testing sets.
4. Convert email text into TF-IDF features.
5. Train the Multinomial Naive Bayes model.
6. Evaluate the model using accuracy.
7. Predict the probability of a new email.

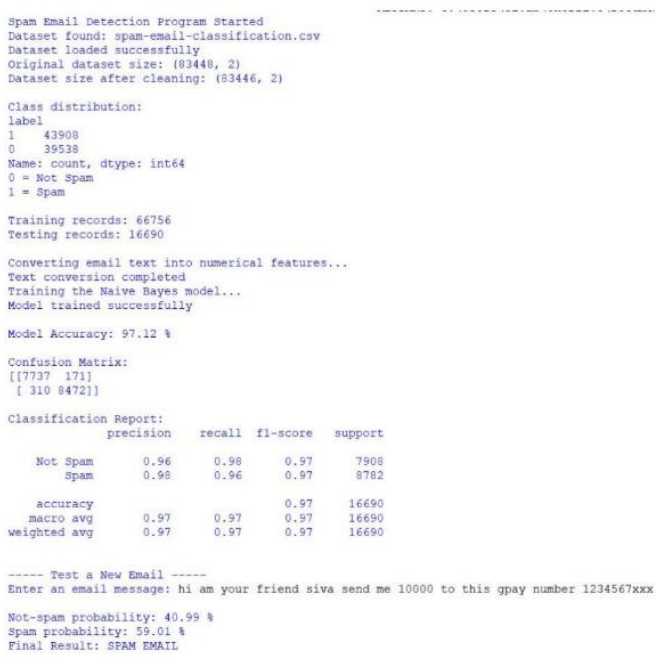
Python Code

```
from pathlib import Path

import pandas as pd

from sklearn.feature_extraction.text import TfidfVectorizer
```

```
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB

path = Path(__file__).parent / "spam-email-classification.csv"
data = pd.read_csv(path, encoding_errors="ignore",
                   on_bad_lines="skip")

data = data[["text", "label"]].dropna()
data = data.drop_duplicates(subset=["text"])
data["label"] = pd.to_numeric(data["label"], errors="coerce")
data = data.dropna()
data["label"] = data["label"].astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    data["text"], data["label"], test_size=0.2,
    random_state=42, stratify=data["label"])

vectorizer = TfidfVectorizer(
    stop_words="english", max_features=10000)

X_train = vectorizer.fit_transform(X_train)
X_test = vectorizer.transform(X_test)

model = MultinomialNB()
model.fit(X_train, y_train)

prediction = model.predict(X_test)
print("Accuracy:", round(
```

```
    accuracy_score(y_test, prediction) * 100, 2), "%")
print("Confusion Matrix:\n", confusion_matrix(y_test, prediction))

message = input("Enter an email: ")
message_vector = vectorizer.transform([message])

result = model.predict(message_vector)[0]
probability = model.predict_proba(message_vector)[0]

print("Not-Spam Probability:",
      round(probability[0] * 100, 2), "%")
print("Spam Probability:",
      round(probability[1] * 100, 2), "%")
print("Result:", "SPAM" if result == 1 else "NOT SPAM")
```

Output Screenshot



```
Spam Email Detection Program Started
Dataset found: spam-email-classification.csv
Dataset loaded successfully
Original dataset size: (83448, 2)
Dataset size after cleaning: (83446, 2)

Class distribution:
label
1    43908
0    39538
Name: count, dtype: int64
0 = Not Spam
1 = Spam

Training records: 66756
Testing records: 16690

Converting email text into numerical features...
Text conversion completed
Training the Naive Bayes model...
Model trained successfully

Model Accuracy: 97.12 %

Confusion Matrix:
[[7737  171]
 [ 310 8472]]

Classification Report:
              precision    recall  f1-score   support

    Not Spam       0.96      0.98      0.97      7908
        Spam       0.98      0.96      0.97      8782

    accuracy                           0.97     16690
   macro avg       0.97      0.97      0.97     16690
weighted avg       0.97      0.97      0.97     16690


----- Test a New Email -----
Enter an email message: hi am your friend siva send me 10000 to this gpay number 1234567xxx

Not-spam probability: 40.99 %
Spam probability: 59.01 %
Final Result: SPAM EMAIL
```

Result

The system calculated spam and non-spam probabilities using Multinomial Naive Bayes and successfully classified the entered email.

Q2.

Student-Performance Prediction

Problem Description

The system predicts whether a student will pass or be academically at risk using study hours, attendance, sleep, previous scores, tutoring and physical activity.

- 0 – At Risk
- 1 – Pass

Students with an exam score of 60 or above are labelled as passing.

Dataset Description

The Student Performance Factors dataset contains **6,607 records and 20 columns** related to study habits, attendance and examination performance.

Dataset file: StudentPerformanceFactors.csv
Source: Student Performance Factors – Kaggle

Algorithm

1. Load the student-performance dataset.
2. Select important academic features.
3. Remove missing and duplicate values.
4. Convert exam scores into pass and at-risk classes.
5. Split the data into training and testing sets.
6. Train the Gaussian Naive Bayes model.
7. Predict a new student's performance.

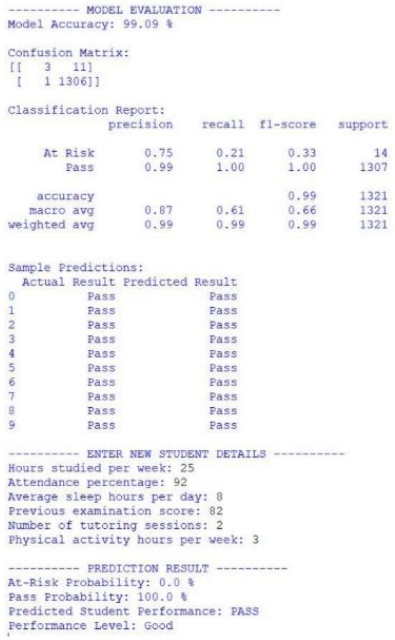
Python Code

```
from pathlib import Path

import pandas as pd

from sklearn.metrics import accuracy_score, confusion_matrix

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import GaussianNB


path = Path(__file__).parent / "StudentPerformanceFactors.csv"
```

```
data = pd.read_csv(path)

features = [
    "Hours_Studied", "Attendance", "Sleep_Hours",
    "Previous_Scores", "Tutoring_Sessions",
    "Physical_Activity"
]

data = data[features + ["Exam_Score"]].copy()

for column in data.columns:
    data[column] = pd.to_numeric(data[column], errors="coerce")

data = data.dropna().drop_duplicates()
data["Performance"] = (data["Exam_Score"] >= 60).astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    data[features], data["Performance"], test_size=0.2,
    random_state=42, stratify=data["Performance"])

model = GaussianNB()
model.fit(X_train, y_train)

prediction = model.predict(X_test)
print("Accuracy:", round(
    accuracy_score(y_test, prediction) * 100, 2), "%")
print("Confusion Matrix:\n", confusion_matrix(y_test, prediction))

values = [
```

```
    float(input("Study hours: ")),
    float(input("Attendance: ")),
    float(input("Sleep hours: ")),
    float(input("Previous score: ")),
    float(input("Tutoring sessions: ")),
    float(input("Physical activity: "))
]

student = pd.DataFrame([values], columns=features)
result = model.predict(student)[0]
probability = model.predict_proba(student)[0]

print("At-Risk Probability:",
      round(probability[0] * 100, 2), "%")
print("Pass Probability:",
      round(probability[1] * 100, 2), "%")
print("Result:", "PASS" if result == 1 else "AT RISK")
```

Output Screenshot



```
---------- MODEL EVALUATION ----------
Model Accuracy: 99.09 %

Confusion Matrix:
[[   3   11]
 [   1 1306]]

Classification Report:
              precision    recall  f1-score   support

     At Risk       0.75      0.21      0.33        14
        Pass       0.99      1.00      1.00      1307

    accuracy                           0.99      1321
   macro avg       0.87      0.61      0.66      1321
weighted avg       0.99      0.99      0.99      1321


Sample Predictions:
  Actual Result Predicted Result
0          Pass             Pass
1          Pass             Pass
2          Pass             Pass
3          Pass             Pass
4          Pass             Pass
5          Pass             Pass
6          Pass             Pass
7          Pass             Pass
8          Pass             Pass
9          Pass             Pass

---------- ENTER NEW STUDENT DETAILS ----------
Hours studied per week: 25
Attendance percentage: 92
Average sleep hours per day: 8
Previous examination score: 82
Number of tutoring sessions: 2
Physical activity hours per week: 3

---------- PREDICTION RESULT ----------
At-Risk Probability: 0.0 %
Pass Probability: 100.0 %
Predicted Student Performance: PASS
Performance Level: Good
```

Result

The model achieved **99.09% accuracy**. However, the at-risk recall was only **21%** because the dataset contained very few at-risk students compared with passing students.

Q3.

Weather Forecasting Using Probability Theory

Problem Description

The system predicts the next weather condition using random variables and conditional probability. The current weather is represented by X_t, and the next weather is represented by X_{t+1}.

$$P(X_{t+1} \mid X_t)$$

The state with the highest transition probability is selected.

Dataset Description

The Weather in Szeged 2006–2016 dataset contains historical temperature, humidity, wind speed, atmospheric pressure and weather-summary values.

Dataset file: weatherHistory.csv
Source: Weather in Szeged 2006–2016 – Kaggle

Algorithm

1. Load and clean the weather dataset.
2. Arrange the records according to time.
3. Convert descriptions into weather states.
4. Create current-state and next-state pairs.
5. Calculate transition probabilities.
6. Accept the current weather state.
7. Predict the state with the highest probability.

Python Code

```
from pathlib import Path

import pandas as pd


path = Path(__file__).parent / "weatherHistory.csv"
```

```
data = pd.read_csv(path)

data["Formatted Date"] = pd.to_datetime(
    data["Formatted Date"], errors="coerce", utc=True)

data = data.dropna(subset=["Formatted Date", "Summary"])
data = data.sort_values("Formatted Date")

def state(summary):
    summary = str(summary).lower()

    if "rain" in summary or "drizzle" in summary:
        return "Rainy"
    if "snow" in summary:
        return "Snowy"
    if "fog" in summary or "mist" in summary:
        return "Foggy"
    if "windy" in summary or "breezy" in summary:
        return "Windy"
    if "cloud" in summary or "overcast" in summary:
        return "Cloudy"
    if "clear" in summary:
        return "Clear"
    return "Other"

data["Current"] = data["Summary"].apply(state)
data["Next"] = data["Current"].shift(-1)
data["Next_Time"] = data["Formatted Date"].shift(-1)
```

```
data["Gap"] = (
    data["Next_Time"] - data["Formatted Date"]
).dt.total_seconds() / 3600

data = data[data["Gap"].between(0.5, 1.5)]
data = data.dropna(subset=["Next"])

table = pd.crosstab(
    data["Current"], data["Next"], normalize="index"
).mul(100)

print("Weather Distribution:")
print(data["Current"].value_counts(normalize=True).mul(100).round(2))

current = input("\nEnter current weather: ").title()
forecast = table.loc[current].sort_values(ascending=False)

print("\nForecast Probabilities:")
print(forecast.round(2))
print("Predicted Weather:", forecast.idxmax())
print("Probability:", round(forecast.max(), 2), "%")
```

Output Screenshot

```
---------- TRANSITION PROBABILITY (%) ----------
Next_State      Clear  Cloudy  Foggy  Rainy  Windy
Current_State
Clear           70.62   26.92   2.21   0.00   0.25
Cloudy           3.90   93.66   1.56   0.02   0.85
Foggy            2.53   17.53  79.82   0.01   0.10
Rainy            0.00   15.18   0.89  83.93   0.00
Windy            2.07   39.16   0.61   0.00  58.16

Available current weather states:
- Clear
- Cloudy
- Foggy
- Rainy
- Windy

Enter the current weather state: cloudy

Forecast probabilities when current weather is Cloudy:
Cloudy: 93.66 %
Clear: 3.9 %
Foggy: 1.56 %
Windy: 0.85 %
Rainy: 0.02 %

---------- FORECAST RESULT ----------
Most likely next weather: Cloudy
Probability: 93.66 %

---------- RAIN RANDOM VARIABLE ----------
P(Rain in next hour): 0.02 %
Expected value E[R]: 0.0002
Variance Var(R): 0.0002

Outcome: Cloudy is the most likely weather condition in the next hour.
```

Result

The system calculated conditional transition probabilities and predicted the next weather state using the outcome with the highest probability.

The limitations include missing measurements, rare weather events, location dependency and changing weather patterns.

Q4.

Banking Fraud Detection Using Conditional Probability

Problem Description

The system uses conditional probability and Gaussian Naive Bayes to identify suspicious banking transactions.

Let:

- F– Fraudulent transaction
- H– High-amount transaction

Bayes' theorem calculates $P(F \mid H)$.

Dataset Description

The Credit Card Fraud Detection dataset contains **284,807 transactions**, including **492 fraudulent transactions**. Class = 0 represents genuine and Class = 1 represents fraud.

Dataset file: creditcard.csv
Source: Credit Card Fraud Detection – Kaggle

Algorithm

1. Load and clean the transaction dataset.

2. Calculate fraud and genuine prior probabilities.
3. Define high-amount transactions.
4. Calculate conditional probabilities.
5. Apply Bayes' theorem.
6. Train Gaussian Naive Bayes.
7. Evaluate and display suspicious transactions.

Python Code

```
from pathlib import Path
import pandas as pd
from sklearn.metrics import (
    accuracy_score, precision_score,
    recall_score, confusion_matrix
)
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB

path = Path(__file__).parent / "creditcard.csv"
data = pd.read_csv(path).dropna().drop_duplicates()

total = len(data)
p_fraud = (data["Class"] == 1).sum() / total
p_genuine = (data["Class"] == 0).sum() / total

limit = data["Amount"].quantile(0.95)
data["High"] = (data["Amount"] >= limit).astype(int)

p_high_fraud = data[data["Class"] == 1]["High"].mean()
p_high_genuine = data[data["Class"] == 0]["High"].mean()
```

```
p_high = (
    p_high_fraud * p_fraud +
    p_high_genuine * p_genuine
)

p_fraud_high = p_high_fraud * p_fraud / p_high

print("P(Fraud):", round(p_fraud * 100, 4), "%")
print("P(Fraud | High):", round(p_fraud_high * 100, 4), "%")

features = ["Time", "Amount"] + [
    "V" + str(i) for i in range(1, 29)
]

X_train, X_test, y_train, y_test = train_test_split(
    data[features], data["Class"], test_size=0.2,
    random_state=42, stratify=data["Class"])

model = GaussianNB()
model.fit(X_train, y_train)

prediction = model.predict(X_test)
probability = model.predict_proba(X_test)[:, 1]

print("Accuracy:", round(
    accuracy_score(y_test, prediction) * 100, 2), "%")
print("Precision:", round(
    precision_score(y_test, prediction,
                    zero_division=0) * 100, 2), "%")
```

```
print("Recall:", round(
    recall_score(y_test, prediction,
                 zero_division=0) * 100, 2), "%")
print("Confusion Matrix:\n", confusion_matrix(y_test, prediction))


result = pd.DataFrame({
    "Amount": X_test["Amount"],
    "Fraud Probability": probability * 100
})


print("\nMost Suspicious Transactions:")
print(result.sort_values(
    "Fraud Probability", ascending=False).head())
```

Output Screenshot

```
---------- MODEL EVALUATION ----------
Accuracy: 99.27 %
Fraud Precision: 13.77 %
Fraud Recall: 64.21 %
Fraud F1 Score: 22.68 %

Confusion Matrix:
[[56269   382]
 [   34    61]]

Classification Report:
              precision    recall  f1-score   support

     Genuine       1.00      0.99      1.00     56651
       Fraud       0.14      0.64      0.23        95

    accuracy                           0.99     56746
   macro avg       0.57      0.82      0.61     56746
weighted avg       1.00      0.99      1.00     56746


---------- MOST SUSPICIOUS TRANSACTIONS ----------
 Amount Actual_Result  Fraud_Probability Predicted_Result
  45.51         Fraud              100.0       Suspicious
1000.00       Genuine              100.0       Suspicious
6454.74       Genuine              100.0       Suspicious
 380.00       Genuine              100.0       Suspicious
   1.00       Genuine              100.0       Suspicious
   2.29       Genuine              100.0       Suspicious
   1.00         Fraud              100.0       Suspicious
  75.86         Fraud              100.0       Suspicious
 523.35       Genuine              100.0       Suspicious
 723.21         Fraud              100.0       Suspicious

Program completed successfully
```

Result

The model calculated fraud probabilities and identified suspicious transactions. Precision and recall are more meaningful than accuracy because fraudulent transactions are rare.

Q5.

Medical Diagnosis Using Bayes Decision Theory

Problem Description

The system predicts whether a tumour is benign or malignant using Gaussian Naive Bayes and Bayes minimum-risk decision theory.

A false-negative diagnosis is more dangerous than a false positive.

- False-positive cost = 1
- False-negative cost = 5

Dataset Description

The Breast Cancer Wisconsin Diagnostic Dataset contains **569 patient records**, including **357 benign** and **212 malignant** cases.

Dataset file: data.csv
Source: Breast Cancer Wisconsin Dataset – Kaggle

Algorithm

1. Load and clean the medical dataset.
2. Convert diagnosis labels into 0 and 1.
3. Split the data into training and testing sets.
4. Train Gaussian Naive Bayes.
5. Calculate posterior probabilities.
6. Calculate the risk of each decision.
7. Select the diagnosis with minimum risk.

Python Code

```
from pathlib import Path
import numpy as np
import pandas as pd
from sklearn.metrics import (
    accuracy_score, recall_score, confusion_matrix
)
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
```

```
path = Path(__file__).parent / "data.csv"
data = pd.read_csv(path)

data = data.dropna(axis=1, how="all")
data = data.drop(columns=["id"], errors="ignore")
data = data.drop(columns=[
    column for column in data.columns
    if column.lower().startswith("unnamed")
], errors="ignore")

data["diagnosis"] = data["diagnosis"].map({
    "B": 0, "M": 1
})

data = data.dropna().drop_duplicates()

X = data.drop(columns=["diagnosis"])
y = data["diagnosis"].astype(int)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

model = GaussianNB()
model.fit(X_train, y_train)

probability = model.predict_proba(X_test)
classes = list(model.classes_)
```

```
p_benign = probability[:, classes.index(0)]
p_malignant = probability[:, classes.index(1)]

fp_cost, fn_cost = 1, 5

standard = (p_malignant >= 0.50).astype(int)

risk_malignant = fp_cost * p_benign
risk_benign = fn_cost * p_malignant

bayes = np.where(
    risk_malignant <= risk_benign, 1, 0)

standard_matrix = confusion_matrix(y_test, standard)
bayes_matrix = confusion_matrix(y_test, bayes)

standard_cost = (
    standard_matrix[0, 1] * fp_cost +
    standard_matrix[1, 0] * fn_cost
)

bayes_cost = (
    bayes_matrix[0, 1] * fp_cost +
    bayes_matrix[1, 0] * fn_cost
)

print("Bayes Threshold:", round(
    fp_cost / (fp_cost + fn_cost) * 100, 2), "%")
print("Accuracy:", round(
```

    accuracy_score(y_test, bayes) * 100, 2), "%")

print("Sensitivity:", round(

    recall_score(y_test, bayes) * 100, 2), "%")

print("Confusion Matrix:\n", bayes_matrix)

print("Standard Cost:", standard_cost)

print("Bayes Cost:", bayes_cost)

Output Screenshot

```
---------- BAYES MINIMUM-RISK DECISION ----------
Accuracy: 93.86 %
Malignant Precision: 97.3 %
Sensitivity: 85.71 %
Specificity: 98.61 %
Decision cost: 31

Confusion Matrix:
[[71  1]
 [ 6 36]]

Classification Report:
              precision    recall  f1-score   support

      Benign       0.92      0.99      0.95        72
   Malignant       0.97      0.86      0.91        42

    accuracy                           0.94       114
   macro avg       0.95      0.92      0.93       114
weighted avg       0.94      0.94      0.94       114


---------- COST COMPARISON ----------
Standard decision cost: 35
Bayes decision cost: 31
Bayes decision theory produced a lower medical decision cost.

---------- HIGH-RISK PATIENT PREDICTIONS ----------
Actual_Diagnosis  Benign_Probability  Malignant_Probability  Risk_Decide_Benign  Risk_Decide_Malignant Bayes_Decision
       Malignant                 0.0                  100.0                 5.0                    0.0      Malignant
       Malignant                 0.0                  100.0                 5.0                    0.0      Malignant
       Malignant                 0.0                  100.0                 5.0                    0.0      Malignant
       Malignant                 0.0                  100.0                 5.0                    0.0      Malignant
       Malignant                 0.0                  100.0                 5.0                    0.0      Malignant
       Malignant                 0.0                  100.0                 5.0                    0.0      Malignant
       Malignant                 0.0                  100.0                 5.0                    0.0      Malignant
       Malignant                 0.0                  100.0                 5.0                    0.0      Malignant
       Malignant                 0.0                  100.0                 5.0                    0.0      Malignant
       Malignant                 0.0                  100.0                 5.0                    0.0      Malignant

Program completed successfully
Educational model only - not for clinical diagnosis.
```

Result

The Bayes model achieved **93.86% accuracy**. It reduced the malignant threshold to **16.67%**, improved sensitivity from **83.33% to 85.71%**, and reduced decision cost from **35 to 31**.
